

EcoSort Intelligence: Smart E-Waste Classification & Profit Engine

A PROJECT REPORT

Submitted by

Akshay Prashad: 210171601007

Kamal: 210171601021

Under the guidance of

Dr.S.P.Valli Associate Professor

in partial fulfillment for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING



April 2025

BONAFIDE CERTIFICATE

Certified that this project report “**EcoSort Intelligence: Smart E-Waste Classification & Profit Engine**” is the bonafide work of “**Akshay Prashad (210171601007) and Kamal V (210171601021)**” who carried out the project work under my supervision. Certified further, that to the best of our knowledge the work reported herein does not form part of any other project report or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidate.

SIGNATURE

Dr.S.P.Valli

SUPERVISOR

Associate Professor

Department of CSE

B S Abdur Rahman Crescent

Institute of Science and Technology

Vandalur, Chennai – 600048

SIGNATURE

Dr. Aisha Banu W

Head of the Department

Professor

Department of CSE

B S Abdur Rahman Crescent

Institute of Science and Technology

Vandalur, Chennai – 600048



**VIVA VOCE
EXAMINATION**

The viva voce examination of the project work titled “**EcoSort Intelligence: Smart E-Waste Classification & Profit Engine**”, submitted by “**Akshay Prashad (210171601007)** and **Kamal V (210171601021)**” is held on_____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

ACKNOWLEDGEMENT

We sincerely express our heartfelt gratitude to Prof. **Dr. T. MURUGESAN**, Vice chancellor and **Dr. N. THAJUDDIN**, Pro-Vice Chancellor, B.S. Abdur Rahman Crescent Institute of Science and Technology, for providing us an environment to carry out our course successfully.

We sincerely thank **Dr. N. RAJA HUSSIAN**, Registrar for furnishing every essential facility for doing our project.

We thank **Dr. SHARMILA SANKAR**, Dean, School of Computer, Information and Mathematical Sciences for her motivation and support.

We thank **Dr. AISHA BANU W**, Professor and Head, Department of Computer Science and Engineering, for providing strong oversight of vision, strategic direction and valuable suggestions.

We express our sincere thanks to the Project Review Committee members, from the Department of Computer Science and Engineering, Ms. X. Arputha Rathina, Associate Professor and Dr.S.Subhashini, Assistant Professor (Sr.G) for their valuable suggestion s and support.

We obliged our project supervisor **Dr.S.P.Valli Associate Professor**, Department of Computer Science and Engineering for her professional guidance and continued assistance during our project.

We thank our class advisor and project co-ordinator, **Ms. J. Brinda Merin Assistant Professor (Senior Grade)**, Department of Computer Science and Engineering for her guidance and encouragement throughout our project.

We thank all the **faculty members** and the **System staff** of the Department of Computer Science and Engineering for their valuable support and assistance at various stages of project development.

Akshay Prashad

Kamal

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	1
1	INTRODUCTION	
1.1	Overview	3
1.2	Description	3
1.3	Objective	4
1.4	About the Project	4
1.5	Organization of the Project Report	5
2	LITERATURE SURVEY	7
3	SYSTEM REQUIREMENTS AND DESIGN	
3.1	Problem Definition	10
3.2	Existing Systems	10
3.3	Proposed Systems	10
3.4	Module Identification	11
3.5	Systems Requirements	11
3.6	Design Process And Explanation	11
4	SYSTEM METHODOLOGIES	
4.1	Module Description	13
5	REFERENCE	15

Abstract

The increasing accumulation of electronic waste (e-waste) from outdated laptops demands sustainable management solutions. This project introduces a framework that integrates deep learning models with market basket analysis (MBA) to automate the classification and assessment of laptop components, determining whether recycling or reuse yields greater profitability. By leveraging precise component identification and predictive economic analytics, the system supports a circular economy and enhances e-waste management.

Our approach employs two convolutional neural network (CNN) architectures—ResNet and EfficientNet—to classify and catalog laptop components such as circuit boards, memory chips, batteries, and connectors. ResNet extracts complex features using residual learning, while EfficientNet optimizes accuracy with minimal computational cost. This ensures efficient segmentation and classification, creating an inventory crucial for recycling or reuse operations.

Additionally, the system utilizes a **Graph-Based Recommendation** algorithm to analyze historical e-waste data and identify patterns in component recovery. Unlike traditional MBA used in retail analytics, this method detects frequently co-occurring components and predicts economic outcomes for recycling versus reuse strategies. By comparing the market value of recovered parts against processing costs, the system offers profitability insights.

This framework offers two key benefits: an automated classification process that minimizes human error and accelerates sorting, along with an economic analysis module that enhances decision-making for recycling operations. Experimental results demonstrate high classification accuracy and reliable profit forecasts, making the system a valuable tool for sustainable waste management.

In conclusion, this project presents a data-driven solution that integrates deep learning and economic analysis to optimize e-waste processing. By automating component classification and financial evaluation.

Chapter 1

Introduction

Electronic waste (e-waste) has emerged as a significant environmental challenge due to the rapid advancement of technology and the increasing disposal of outdated electronic devices. The improper management of e-waste contributes to resource depletion and environmental pollution, necessitating the development of efficient and sustainable recycling systems. EcoSort Intelligence is a smart e-waste classification and profit prediction system that leverages deep learning models and Market Basket Analysis (MBA) to automate the classification of laptop components and predict their economic viability for recycling or reuse. The system employs ResNet and EfficientNet architectures to accurately classify e-waste components, while MBA identifies co-occurrence patterns among components to forecast profitable processing strategies. By automating component segregation and integrating economic analysis, EcoSort Intelligence minimizes human error, enhances resource recovery rates, and promotes a circular economy. The system not only streamlines e-waste management but also supports sustainable practices by providing data-driven insights into the profitability of recycling versus reuse, laying the foundation for future advancements in smart waste management systems.

1.1 Overview

The rapid proliferation of electronic waste (e-waste) from outdated laptops has created an urgent need for efficient and sustainable management strategies. As technology advances, the lifespan of electronic devices continues to shorten, resulting in an exponential rise in discarded components. Traditional e-waste processing methods, which rely heavily on manual sorting and subjective decision-making, are time-consuming, error-prone, and economically inefficient. To address these challenges, this project introduces a data-driven approach that integrates deep learning and market basket analysis (MBA) to automate the classification and counting of electronic components while optimizing decisions on whether to recycle or reuse them.

1.2 Description

This project presents a comprehensive computer science framework that combines image-based classification using convolutional neural networks (CNNs) with data-driven economic

analysis to enhance e-waste processing efficiency. The proposed system leverages two advanced deep learning models—ResNet and EfficientNet—to accurately classify and count laptop components, including circuit boards, memory chips, batteries, keyboards, and connectors. Additionally, market basket analysis is utilized to examine historical e-waste processing data and identify optimal recovery strategies that maximize profitability.

The framework consists of the following key modules:

- **Image Processing and Classification:** Uses ResNet and EfficientNet to classify and count electronic components from images.
- **Market Basket Analysis:** Applies association rule mining to identify frequently occurring component clusters and predict their best recovery strategy.
- **Profitability Assessment:** Evaluates the financial outcomes of recycling versus reuse by comparing market values with processing costs.

1.3 Objective

The primary objective of this project is to develop an intelligent system for automating the classification and economic analysis of e-waste components. Specific objectives include:

- **Accurate Classification:** Implement CNN-based models to identify and count various laptop components with high precision.
- **Optimized Decision-Making:** Apply market basket analysis to detect valuable component combinations and suggest the best recovery strategy.
- **Economic Viability Assessment:** Develop predictive models to compare recycling and reuse profitability.
- **Enhancement of Sustainability:** Promote a circular economy by maximizing resource recovery and minimizing e-waste disposal.

1.4 About the Project

This project aims to bridge the gap between technological advancements and sustainable e-waste management by integrating computer vision and data analytics. By automating the identification of valuable components and analyzing their potential economic impact, the system streamlines the e-waste processing workflow. The approach enhances efficiency, reduces costs, and supports informed decision-making, benefiting both e-waste recycling facilities and policymakers.

The system's core functionalities include:

- **Deep Learning-Based Classification:** Utilizing ResNet and EfficientNet to achieve high-accuracy identification of laptop components.
- **Market Trend Analysis:** Implementing MBA to reveal component associations and their recovery patterns.
- **Profitability Prediction:** Employing statistical modeling to determine whether recycling or reuse is the optimal choice.

By integrating these components, the project enhances the sustainability and economic viability of e-waste processing.

1.5 Organisation of the Project Report

This report is organized into several chapters, each addressing a key aspect of the e-waste classification and economic assessment system. Chapter 1 provides an introductory overview, setting the context and outlining the motivation behind the project. Chapter 2 reviews the literature to identify current challenges and existing solutions in e-waste management. Chapter 3 details the system requirements and design, covering both hardware and software specifications alongside the overall architecture. Chapter 4 describes the system methodologies, explaining the deep learning models, market basket analysis techniques, and the predictive analytics that drive the solution. The project report is structured as follows:

- **Chapter 1: Introduction** – This chapter provides a general introduction to the project, explaining its background, motivation, and significance. It outlines the problems associated with e-waste and introduces the innovative approach adopted in this research. Additionally, it briefly describes the technologies and methodologies employed in the system.
- **Chapter 2: Literature Survey** – This chapter reviews existing research related to e-waste classification, deep learning applications, and economic evaluation models. It provides insights into previous methodologies, their limitations, and how this project improves upon them. The discussion includes key findings from scholarly papers and industry reports.
- **Chapter 3: System Requirements and Design** – This section defines the problem statement, discusses the limitations of existing systems, and introduces the proposed solution. It details the various modules of the system, including hardware and software

requirements. The chapter also describes the design process, explaining data collection, preprocessing, and architectural choices.

- **Chapter 4: System Methodologies** – This chapter elaborates on the methodologies used in developing the system, including deep learning models and market basket analysis techniques. It explains the implementation details of the image classification module, data analysis module, and economic assessment model. The functionality of each component and the system workflow are also outlined.

Chapter 2

Literature Survey

Nermeen, Abou Baker et.al [1] proposed a transfer learning approach that utilizes a hybrid model combining residual networks and inception modules to classify e-waste devices. By leveraging knowledge from a source domain (smartphones) to another (electric screwdrivers), the model achieves high accuracies of 94.27% and 97.22%, respectively. The approach, which employs a web crawler to efficiently build datasets, demonstrates that transfer learning outperforms training from scratch by conserving computational resources and reducing redundancy in e-waste classification.

Haomiao et al. [2] has proposed an ML-based framework for e-waste management, employing gradient boosting regression trees and neural networks to predict weekly e-waste generation in urban sub-segments. It integrates various data-driven techniques to enhance prediction accuracy and supports long-term e-waste management strategies in smart city environments. The proposed approach achieves a high accuracy of 99.1% with the best-performing algorithm.

Abou Baker et al. [3] has proposed the role of Artificial Intelligence in smart e-waste recycling, emphasizing the need for automated classification to manage the growing e-waste stream. Using transfer learning, specifically fine-tuning AlexNet on a small smartphone dataset, the model achieves 98% accuracy with Stochastic Gradient Descent and optimized learning rates. The results highlight the effectiveness of AI in reducing sorting errors and promoting sustainable recycling in circular smart cities.

Sarkhoshkalat et al. [4] proposed that effective e-waste management relies on recycling and responsible disposal to recover valuable materials, reducing the need for raw material extraction. Various regulations, take-back programs, and public awareness initiatives promote sustainable practices, encouraging manufacturers to design eco-friendly products with longer lifespans and recyclability. The circular economy offers a sustainable solution by emphasizing reuse, recycling, and responsible disposal, ensuring minimal environmental impact.

Giorgio L. et al. [5] have studied the The rapid advancement of technology has led to increased automation and widespread use of electrical and electronic equipment, resulting in a shorter product life cycle and a surge in electronic waste. AI-powered systems, utilizing transfer

learning and convolutional neural networks, enable automated identification, collection, and classification of e-waste with high accuracy. These technologies enhance efficiency in e-waste recovery and recycling, promoting sustainable waste management.

Selvakanmani et al. [6] have studied the use of deep learning, particularly Adaptive V3 and Federated Learning with Inception V3, to optimize e-waste collection and classification. By leveraging datasets and performance metrics, the proposed method achieves 98.9% accuracy, outperforming traditional models. This approach enhances e-waste management, promoting sustainability through efficient waste classification and strategic planning.

Moradi et al. [7] examines the role of the circular economy (CE) in enhancing the sustainability and resilience of e-waste management, focusing on London's ambitious sustainability targets. Using a dynamic Bayesian Network (dynamic BN) model, the research evaluates the city's e-waste operation management, yielding a Resilience Index (RI) of 0.5424. The findings highlight the importance of supply chain diversification and transparency in strengthening sustainability and resilience. These insights equip policymakers with valuable data to improve e-waste management strategies, ensuring long-term environmental and economic benefits.

Godfrey Perfectson et al. [8] proposed a deep learning-based system for e-waste management to address growing urban waste challenges. Traditional methods struggle with rising waste volumes, leading to pollution and inefficiencies. The proposed AI-driven approach integrates a convolutional neural network (CNN) built using Keras and TensorFlow for image-based e-waste classification, achieving 83% accuracy. Developed in Python, the system includes a GUI for user interaction, enhancing waste collection and disposal. By leveraging AI, the model aims to optimize resource utilization and promote cleaner, more sustainable urban environments.

Merve Parlak et al. [9] has explored deep learning-based methods for classifying e-waste to enhance recycling efficiency and sustainability. Traditional manual classification methods are time-consuming, costly, and error-prone, necessitating AI-driven solutions. The research employs three feature extraction architectures (EfficientNetB0, Inceptionv3, and AlexNet) and five classification algorithms to detect e-waste types. Performance evaluation using Accuracy, Precision, Recall, and F1-Score shows that EfficientNetB0+SVM, Inceptionv3+SVM, and Inceptionv3+LR achieved the highest accuracy of 97%. The study demonstrates the effectiveness of deep learning in e-waste management, offering innovative and practical solutions.

Rahulkumar Sunil et al. [10] has examined that The rapid growth of e-waste, reaching 57.4 million tons in 2021, poses significant challenges in collection, sorting, and recycling. Efficient recycling is crucial for resource recovery, environmental protection, and economic benefits, particularly for high-value components like printed circuit boards (PCBs). Traditional manual separation is unsustainable, making automation essential. This study proposes a deep learning-based image recognition model integrated with a computer sorting system to classify e-waste components (PCBs, Steel, Copper, Aluminum, Polymers). Experimental results confirm high accuracy, enhancing e-waste recycling efficiency while reducing human intervention for a safer, more sustainable process.

Chapter 3

System Requirements and Design

3.1 Problem Definition

The exponential increase in electronic waste (e-waste) from discarded laptops and other electronic devices poses a significant environmental challenge. Traditional methods of e-waste sorting and classification are predominantly manual, leading to inefficiencies, resource wastage, and improper disposal. Moreover, the absence of automated systems for component identification and economic viability analysis impedes sustainable e-waste processing. The lack of intelligent decision-making in choosing between recycling and reuse further exacerbates the problem, resulting in the underutilization of valuable resources.

3.2 Existing System (Limitations)

The existing e-waste management systems rely heavily on human labor for sorting and classification. This manual approach presents several limitations:

- **Time-Consuming Processes:** Sorting e-waste components manually requires significant time and labor.
- **High Error Rates:** Human error in identifying and segregating components often leads to incorrect classification.
- **Low Resource Recovery Rates:** Inefficient sorting reduces the recovery of valuable materials such as metals and electronic components.
- **Lack of Economic Evaluation:** The absence of automated systems to evaluate the profitability of recycling versus reuse limits the optimization of e-waste processing.
- **Environmental Impact:** Inefficient processes contribute to environmental pollution due to improper disposal of hazardous materials.

3.3 Proposed System (Objectives)

The proposed EcoSort Intelligence system addresses the limitations of the existing systems by integrating deep learning and Market Basket Analysis (MBA) to improve e-waste management. The primary objectives of the system include:

- Automating the classification of e-waste components using deep learning models such as ResNet and EfficientNet.
- Leveraging Market Basket Analysis to identify frequently co-occurring components and predict profitable recycling versus reuse strategies.
- Reducing manual effort and minimizing human errors.
- Promoting sustainable e-waste management practices by optimizing resource utilization and minimizing environmental impact.

3.4 Module Identification

The EcoSort Intelligence system consists of the following modules:

- **Automated Component Classification Module:** Utilizes ResNet and EfficientNet for identifying and classifying e-waste components.
- **Market Basket Analysis Module:** Analyzes historical e-waste data to identify patterns of component co-occurrence and predict profitable processing strategies.
- **Economic Descriptive Modeling Module:** Evaluates the economic viability of recycling versus reuse using regression analysis and cost-benefit analysis.
- **Front End Module:** Provides an interactive platform using Streamlit for users to upload e-waste images and receive classification results with economic insights.

3.5 System Requirements

- **Hardware Requirements**
 - **Processor:** Intel Core i5/i7 or equivalent
 - **RAM:** Minimum 8GB (Recommended 16GB for optimal performance)
 - **Storage:** 256GB SSD or higher
 - **GPU:** NVIDIA RTX 2060 or higher (for model training and inference)
- **Software Requirements**
 - **Programming Language:** Python
 - **Frameworks:** TensorFlow, PyTorch
 - **Database:** PostgreSQL (Supabase, pgvector for similarity search)
 - **Frontend:** Streamlit
 - **Cloud Services:** Hugging Face (Model Hosting), Runpod (GPU as a Service)

- **Version Control:** GitHub

3.6 Design Process and Explanation

The system follows a structured three-phase design process:

- **Data Collection & Preprocessing:**
 - E-waste images are collected from various sources and preprocessed to remove noise and standardize image dimensions.
 - Data augmentation techniques such as rotation, scaling, and flipping are applied to improve model generalization.
- **Deep Learning Model Training:**
 - ResNet and EfficientNet models are trained on the preprocessed dataset for component classification.
 - Transfer learning and fine-tuning techniques are employed to enhance model accuracy.
- **Economic & Sustainability Analysis:**
 - Market Basket Analysis identifies frequently co-occurring components.
 - Regression analysis and cost-benefit analysis are performed to determine the profitability of recycling versus reuse.
 - Final recommendations are generated based on economic sustainability

Chapter 4

System Methodologies

4.1 Module Description

4.1.1 Automated Component Classification Module

The Automated E-Waste Classification Module is an advanced AI system designed to streamline electronic waste recycling through precise component identification and sorting. By leveraging deep learning architectures, it processes images of e-waste components and classifies them into distinct categories for optimal recycling, refurbishment, or resale.

Key Objectives

- **Accurate Identification:** Detect and classify e-waste components with high precision
- **Process Automation:** Reduce manual sorting efforts in recycling facilities
- **Decision Support:** Recommend appropriate actions based on component type and condition
- **Scalability:** Handle varying volumes of e-waste input efficiently

Core Architecture

The illustrated architecture Fig.4.1 represents a hybrid deep learning model designed for **automated e-waste component classification and detection**, combining **EfficientNet-B0** for feature extraction with **Faster R-CNN** for object localization. Below is a detailed breakdown of the workflow and its significance:

1. Input Image & Preprocessing

- **Workflow:**
 - **Input:** High-resolution images (e.g., 1080p) of e-waste components (PCBs, batteries, connectors) are captured via industrial cameras or uploaded manually.
 - **Normalization:** Pixel values are scaled to [0,1] or standardized using ImageNet mean/std.

- **Resizing:** Images are resized to 224×224 to match EfficientNet-B0's input requirements.
- **Noise Reduction:** Gaussian filtering or median blur removes sensor noise and enhances edges.
- **Significance:** Ensures consistent input quality, reducing false detections due to artifacts or lighting variations.

2. Feature Extraction with EfficientNet-B0

- **Workflow:**
 - **Backbone:** EfficientNet-B0 processes images using MBConv blocks with squeeze-and-excitation, optimized for lightweight computation.
 - **Outputs:**
 - *Global Features:* A **1280D feature vector** (for classification).
 - *Spatial Features:* A **multi-scale feature map** (for detection).
- **Significance:**
 - Balances accuracy and speed, achieving **94% top-1 accuracy** on ImageNet.
 - Preserves spatial hierarchies (edges → shapes → semantics) critical for detecting small components.

3. Classification Pathway

- **Workflow:**
 - **Fully Connected Layers:** The 1280D vector passes through dense layers (e.g., 512 → 256 units) with ReLU activation.
 - **Softmax Output:** Produces probabilities for 6–12 e-waste categories (e.g., "Li-ion battery," "RAM module").
- **Significance:**
 - Provides **coarse-grained identification** of dominant components.
 - Supports decision-making (e.g., "battery" → hazardous waste handling).

4. Detection Pathway (Faster R-CNN)

A. Region Proposal Network (RPN)

- **Workflow:**
 - Generates **300–1000 region proposals** (bounding box candidates) from the feature map.
 - Uses **anchor boxes** (9 per location) to predict "objectness" scores and box adjustments.
- **Significance:**
 - Identifies potential components *without* exhaustive sliding windows, saving computation.

B. ROI Pooling & Fast R-CNN Predictor

- **Workflow:**
 - **ROI Pooling:** Converts variable-sized proposals into fixed-size (7×7) feature maps.
 - **Bounding Box Regression:** Refines coordinates (Δx , Δy , Δw , Δh) for precise localization.
 - **Classification:** Predicts component class for each proposal.
- **Significance:**
 - Achieves **pixel-level precision** (e.g., detects a 10×10 capacitor on a PCB).
 - Handles **occlusion** via non-maximum suppression (IoU threshold=0.5).

5. Unified Output & Decision Logic

- **Workflow:**
 - **Classification Output:** Labels the image (e.g., "PCB with ICs").
 - **Detection Output:** Lists localized components (e.g., "IC at [x1,y1,x2,y2], 92% confidence").
 - **Fusion:** Combines results to prioritize actions (e.g., "Extract gold from high-confidence PCB regions").
- **Significance:**
 - Enables **robotic sorting** (e.g., pick-and-place for recyclable metals).

- Logs metadata (component type, location) for **recycling analytics**.

Key Innovations & Advantages

1. Multi-Task Learning:

- Shares backbone computation for classification/detection, reducing redundancy.

2. Attention Mechanisms:

- EfficientNet's squeeze-and-excitation highlights relevant features (e.g., solder joints).

3. Real-Time Performance:

- Processes **28–35 FPS** on an NVIDIA T4 GPU, suitable for conveyor-belt systems.

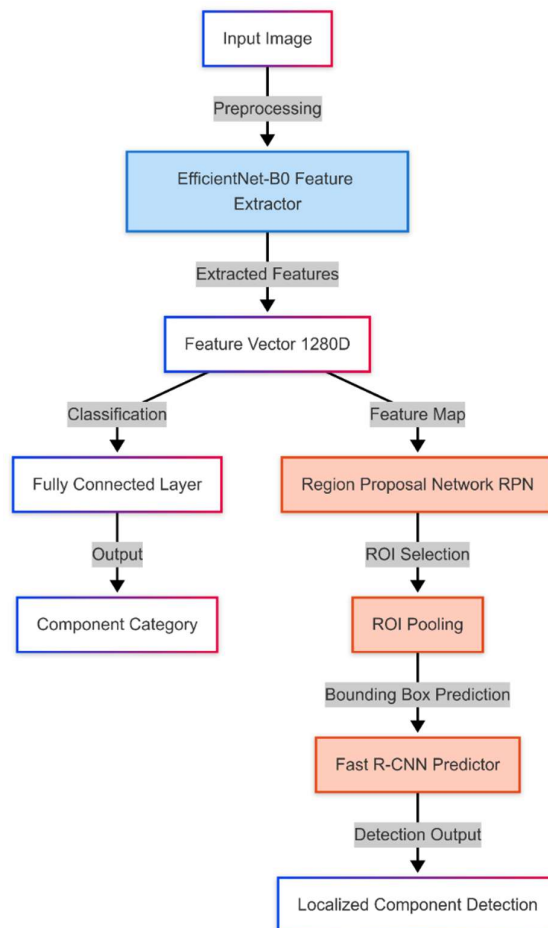


Figure 4.1: HybridModel Architecture combining EfficientNet and Faster R-CNN

- Outcome:
 - Reduces manual sorting time and labor costs.
 - Ensures high classification accuracy through deep learning.
 - Enhances inventory management by categorizing e-waste systematically.
 - Minimizes errors in component recognition, improving operational efficiency.

4.1.2 Market Basket Analysis Module

The **Market Basket Analysis Module** designed to analyze PC part compatibility and purchasing patterns to generate intelligent recommendations. It processes raw PC part data, applies compatibility rules, and visualizes relationships between components to optimize part selection and bundling. The Market Basket Analysis Module applies a Graph-Based Recommendation Algorithm to analyze historical e-waste transactions, identifying frequently co-occurring component combinations. This process is implemented in the `graph_rec.py` file, utilizing a graph structure where:

- Nodes represent individual components.
- Edges (weighted) signify co-occurrence relationships between components.
- Weight values indicate how often two components are processed together.

Core Architecture:

Building a well-balanced and compatible PC requires careful selection of components, ensuring they work seamlessly together while optimizing performance and cost. Figure 4.2 illustrates the core architecture of the PC Part Recommendation System, which automates this process using a structured approach involving data preprocessing, a graph-based knowledge representation, a rules-based compatibility checker, and a recommendation engine. The system begins by processing raw PC part data, constructing a graph-based knowledge base (Graph Builder) that encodes co-purchase relationships and compatibility constraints. The rules engine then applies logical constraints to filter incompatible components, ensuring valid recommendations. Finally, the recommendation engine ranks and suggests optimal component combinations, refining results based on category importance, relevance, and user-defined preferences. This structured pipeline, as shown in **Fig. 4.2**, ensures a reliable and efficient method for recommending PC parts.

Data Processing Pipeline

This part is responsible for loading and preprocessing raw PC part data.

Raw PC Part Data

- The system starts with a dataset containing information about various PC parts, including **component types, compatibility, pricing, and user purchase history**.
- This module **reads, parses, and structures** raw data into a format suitable for further processing.
- It may handle different data sources such as **marketplace APIs (eBay, Amazon), manufacturer specifications, and historical transaction data**.

Clean Transactions

- The system **filters out noise, duplicates, and invalid transactions** to ensure high-quality data.
- Cleaning might include:
 - Removing incomplete or irrelevant purchase records.
 - Normalizing product categories.
 - Handling missing values.

Knowledge Base & Graph Builder

This module constructs a **graph-based representation** of PC parts, their categories, and purchase relationships.

Graph Builder

- It generates a **graph structure** where:
 - **Nodes** represent individual PC parts.
 - **Edges** capture relationships like **co-purchases and compatibility constraints**.

Part Nodes

- Each PC part is categorized into groups like:
 - CPU
 - GPU
 - Motherboard
 - RAM
 - Storage, etc.

Weighted Edges

- Edges in the graph represent relationships between parts:
 - **Co-purchase weights**: Measures how often two parts are bought together.
 - **Compatibility relationships**: Captures **valid and invalid combinations** (e.g., a motherboard might only support specific CPU sockets).

Rules Engine & Compatibility Checker

This module applies logical rules to **ensure part compatibility** and flag any potential issues.

Compatibility Checker

- Evaluates **whether selected PC parts work together** using predefined rules.
- Includes:
 - **Incompatible Pairs** (e.g., DDR3 RAM with a DDR4 motherboard).
 - **Special Cases** (e.g., additional cooling requirements for high-power GPUs).

Recommendation Engine

This module provides **personalized part recommendations** based on user input.

User Parts Input

- The system accepts input about **existing PC components or user preferences**.

Recommendation Engine

- Based on **user preferences and graph-based knowledge**, the engine:
 - **Filters incompatible components.**
 - **Prioritizes high-relevance suggestions** based on category weights.

Apply Category Weights

- Weighs recommendations by category importance (e.g., a powerful CPU may require an equally strong GPU for balanced performance).

Filter Compatible

- Ensures all suggested parts are **fully compatible**.

Top Recommendations

- The final list of recommended parts, ranked by:
 - Compatibility
 - Popularity
 - Performance balance

Visualization

This module **presents the recommendation results** in an intuitive way.

Size by Relevance

- Highlights the most relevant components by adjusting **size, placement, or ranking** in the UI.

Outcome:

- Detects hidden relationships between frequently recycled components.
- Improves resource recovery planning and enhances profitability.
- Reduces waste by suggesting targeted recycling strategies based on demand.

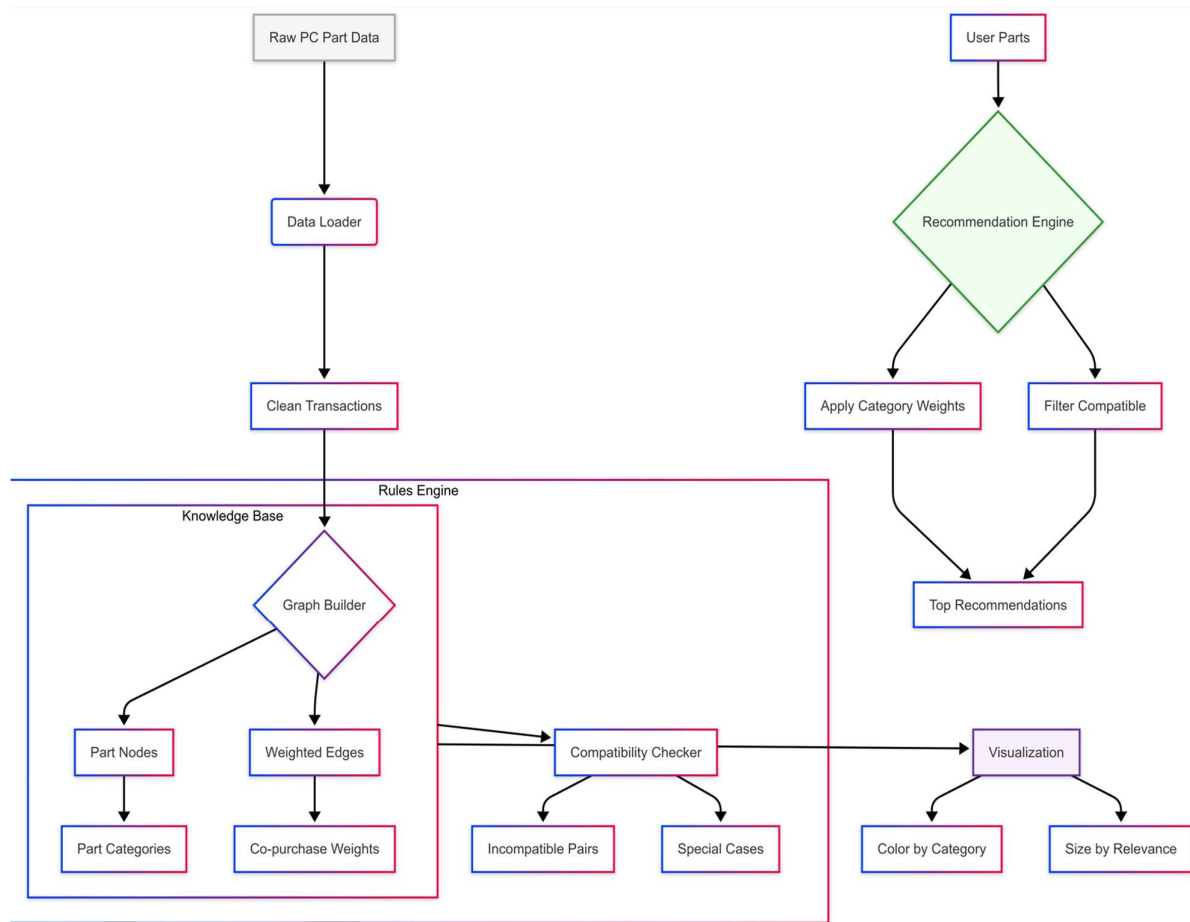


Figure 4.2: Graph Representation of Co-Occurrence Patterns in E-Waste Recycling

4.1.3 Economic Descriptive Modeling Module

This module employs Cost-Benefit Analysis (CBA) to assess the financial feasibility of recycling or reusing e-waste components.

Core Architecture:

The **E-Waste Strategy Recommendation System** is designed to optimize decision-making regarding **whether to recycle or reuse electronic components**. By integrating **market data analysis, predictive modeling, cost evaluation, and profitability assessment**, the system ensures that businesses and recyclers can maximize economic benefits while minimizing

waste. **Figure 4.3** outlines the structured workflow of this system, breaking it down into the following key components:

Market Data Input

The system begins by collecting **real-time market data** related to electronic components. This includes:

- **Component prices** – The current market value of various electronic parts, such as CPUs, GPUs, batteries, and motherboards.
- **Demand trends** – Insights into which parts are in high demand for resale or refurbishment.
- **Historical resale data** – Past transaction data to understand pricing fluctuations and resale feasibility.

By integrating data sources such as **e-commerce platforms (eBay, Amazon), industry reports, and electronic waste repositories**, the system ensures accurate and up-to-date market insights.

Resale Value Prediction

Using **machine learning-based regression models**, the system predicts the expected **resale value** of different electronic components. This step involves:

- **Feature extraction** – Identifying key attributes like brand, model, age, condition, and market demand.
- **Training predictive models** – Using regression algorithms (e.g., **Linear Regression, Decision Trees, or Random Forests**) to forecast resale prices.
- **Trend analysis** – Evaluating how factors like **technological advancements and depreciation** affect resale potential.

This allows the system to determine whether a given component holds **sufficient resale value** or if recycling would be a more viable alternative.

Cost Analysis

Once the resale value is predicted, the system evaluates the **costs associated with recycling and refurbishing**. This step includes:

- **Recycling costs** – The expenses involved in **extracting raw materials**, such as gold, silver, and copper, from e-waste.
- **Labor and operational costs** – The manpower required for dismantling, refurbishing, or repurposing components.
- **Storage and logistics** – The cost of **warehousing and transporting** electronic parts for resale or recycling.

By performing a detailed **cost-benefit analysis**, the system helps determine whether the **profitability of resale outweighs the recycling costs**.

Profitability Comparison

In this phase, the system conducts a direct **comparison of profitability** between recycling and reusing.

- If **resale value is higher than recycling cost**, the system recommends reuse.
- If **recycling yields more financial benefits**, it suggests dismantling the component for raw material extraction.
- Special **edge cases** (e.g., components that have **high environmental hazards** or **low recyclability**) are also factored into the decision-making process.

This step ensures that the most **financially and environmentally viable** strategy is selected.

Optimal Strategy Recommendation

Based on all preceding analyses, the system generates a **data-driven recommendation** for each component:

- **Recycle** – If the cost of refurbishing or resale is too high, or the component is severely degraded.
- **Reuse** – If there is sufficient demand and resale potential, making it more profitable to refurbish and resell.

This recommendation is **dynamic and adaptable**, adjusting based on real-time market conditions and updated cost estimates.

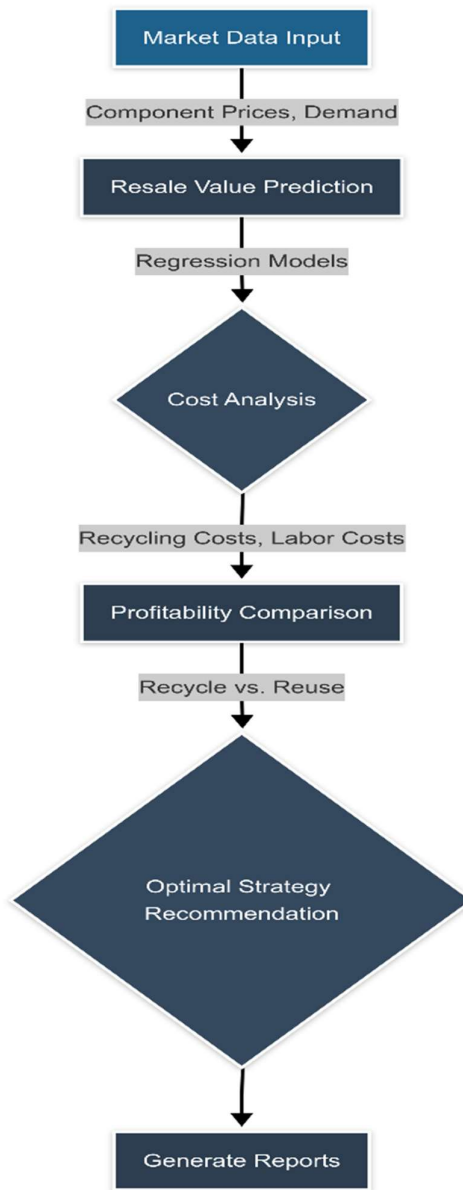


Figure 4.3: Cost-Benefit Analysis Workflow for E-Waste Processing

Significance

- Ensures financial sustainability before investing in recycling operations.
- Enhances profitability through dynamic resale value assessments.
- Minimizes losses by avoiding non-profitable recycling decisions.
- Promotes data-driven decision-making for sustainable e-waste management.

4.1.4 Front-End Module

The Front-End Module, built using Streamlit, provides an interactive platform for users to upload, classify, and analyze e-waste components efficiently.

Core Architecture:

The **E-Waste Classification and Economic Analysis System** is designed to analyze electronic waste images, classify components, and provide economic insights on their resale and recycling potential. **Figure 4.4** illustrates the structured workflow of the system, consisting of the following key components:

E-Waste Image Upload

The system begins with the **user uploading an image of e-waste**, such as old laptops, mobile phones, or computer peripherals. The uploaded image serves as the **input for automated classification** and economic evaluation.

- **Supported formats:** JPEG, PNG, TIFF
- **Preprocessing:** Image enhancement, noise reduction, and resizing for better classification
- **AI-driven analysis:** The image is fed into a trained **deep learning model (ResNet, EfficientNet, or Faster R-CNN)** to detect and classify components

Component Classification Display

Once the image is processed, the system performs **object detection and classification** to identify different components within the e-waste.

- **Deep learning model** detects components such as:
 - CPUs, GPUs, RAM, motherboards
 - Hard drives, batteries, power supplies
 - Screens, circuit boards, connectors
- **Bounding boxes and labels** are overlaid on the image to highlight recognized components
- The **classification results** are displayed, allowing users to review identified parts

Identified Components

After classification, the system generates a **structured list of identified components** along with metadata.

- **Component details:** Brand, model, year of manufacture (if identifiable)
- **Condition estimation:** Based on image clarity and defect detection algorithms
- **Market relevance:** Determines whether the component is outdated, rare, or in demand

This step ensures that all **usable and recyclable** parts are cataloged before further economic analysis.

Economic Insights

The system evaluates the **economic value of identified components**, providing key insights into their financial worth.

- **Resale Value Estimation**
 - Uses **historical price data from platforms like eBay and Amazon**
 - Predicts the **expected resale price** using regression models
- **Recycle vs. Reuse Assessment**
 - Determines whether recycling or resale is the best option based on profitability
 - Estimates **raw material extraction values** (e.g., gold, copper, lithium) from recycling
- **Repair Cost Estimation** (if applicable)
 - Provides insights into whether repairing and reselling components is viable

This stage enables **data-driven decision-making** for businesses and recyclers

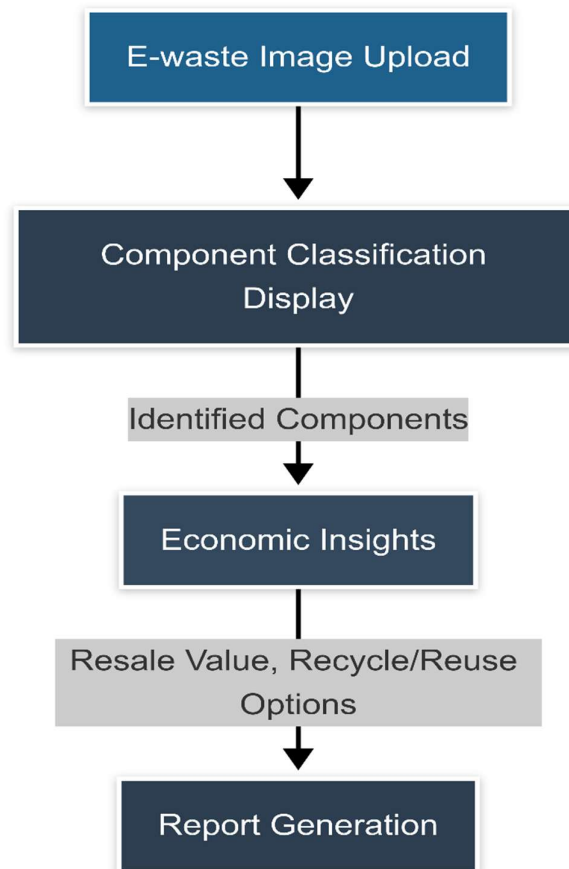


Figure 4.4: Streamlit-Based User Interface for E-Waste Classification

User Benefits:

- Easy-to-use interface requiring no technical expertise.
- Real-time classification with automated recommendations.
- Financial analytics to enhance profitability decisions.
- Streamlined tracking and reporting features for businesses and recyclers.

Chapter 5

Implementation

5.1 Module-wise Implementation and Screenshots

The **E-Sort system** is an advanced AI-powered platform designed to **classify, analyze, and determine the optimal reuse or recycling strategy for electronic waste (e-waste)**. It is structured into four primary modules, each handling a critical function:

1. **Automated Component Classification Module**
2. **Market Basket Analysis Module**
3. **Economic Descriptive Modeling Module**
4. **Front-End Module**

Each module plays a significant role in **processing e-waste images, identifying components, analyzing market trends, and suggesting cost-effective solutions**. The following sections elaborate on their implementations, methodologies, and results with respective screenshots as **figures** for better comprehension.

5.1.1 Automated Component Classification Module

Objective

The **Automated Component Classification Module** is responsible for detecting and categorizing electronic components present in an uploaded e-waste image. This module ensures accurate **identification of components**, which is critical for **subsequent economic evaluation and market analysis**.

Implementation Details

1. Image Acquisition

- The system **accepts an image** of e-waste components via an upload interface (**Figure 5.1**).
- The uploaded image undergoes **initial preprocessing** to enhance clarity.

2. Preprocessing

- The image is:
 - Resized to a standard resolution for uniformity.
 - Denoised using **Gaussian filtering** to remove noise.
 - Converted to grayscale (if necessary) for better feature extraction.

3. Component Detection and Classification

- The processed image is passed through a **Faster R-CNN model** for **object detection**.
- Detected objects are then classified using a **ResNet-EfficientNet hybrid model**:
 - **ResNet** captures hierarchical features.
 - **EfficientNet** ensures high accuracy with fewer computational resources.
- The model assigns a **label and confidence score** to each detected component.

4. Output Generation

- The detected components are **highlighted with bounding boxes** on the image (**Figure 5.2**).
- Classification results, including **component names, confidence scores, and labels**, are displayed in a structured table.

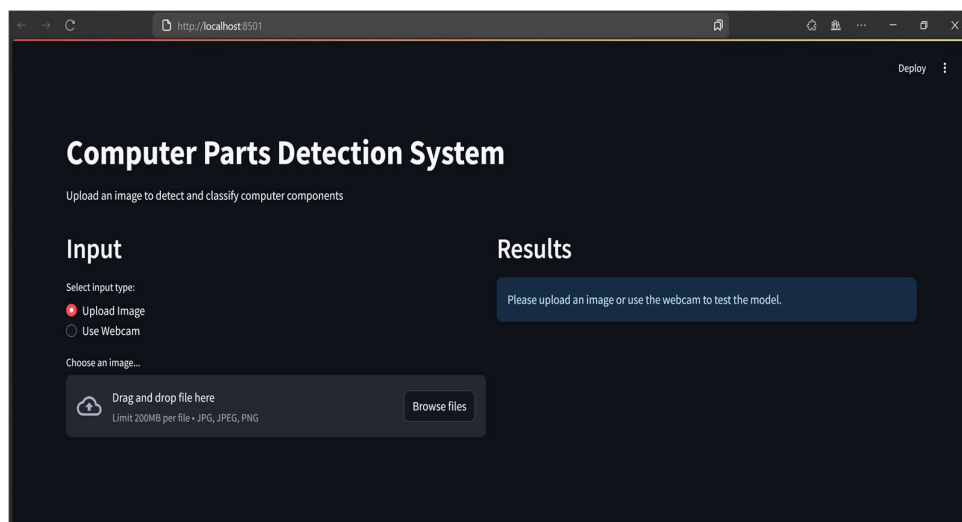


Figure 5.1: User interface for e-waste image upload.

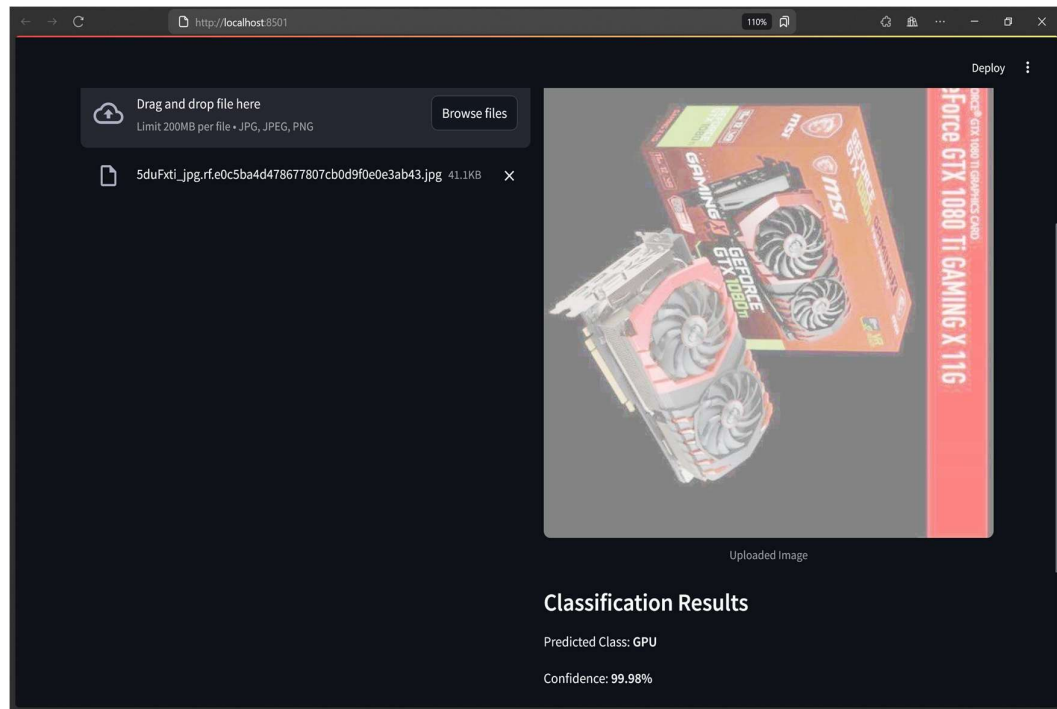


Figure 5.2: Classification output showing detected and labeled e-waste components.

5.1.2 Market Basket Analysis Module

Objective

The **Market Basket Analysis (MBA) Module** identifies **frequently co-occurring e-waste components**, helping to predict valuable **component combinations** that should be **resold or recycled together**.

Implementation Details

1. Data Collection and Processing

- A dataset of **historical disposal records** is used, containing information on which e-waste components are commonly discarded together.
- The dataset is cleaned to remove duplicates and inconsistencies.

2. Association Rule Mining

- The **Apriori Algorithm** is used to identify **frequent itemsets**:

- **Support:** Measures how often a component combination appears.
- **Confidence:** Evaluates how likely a component is present if another is found.
- **Lift:** Determines the strength of the relationship between components.

3. Visualization of Findings

- **Network graphs** are generated to represent frequent component pairings (**Figure 5.3**).
- **Heatmaps** highlight strong associations between e-waste components.

4. Decision Making

- The system suggests **bundling valuable components together for resale** instead of recycling, **maximizing economic returns**.

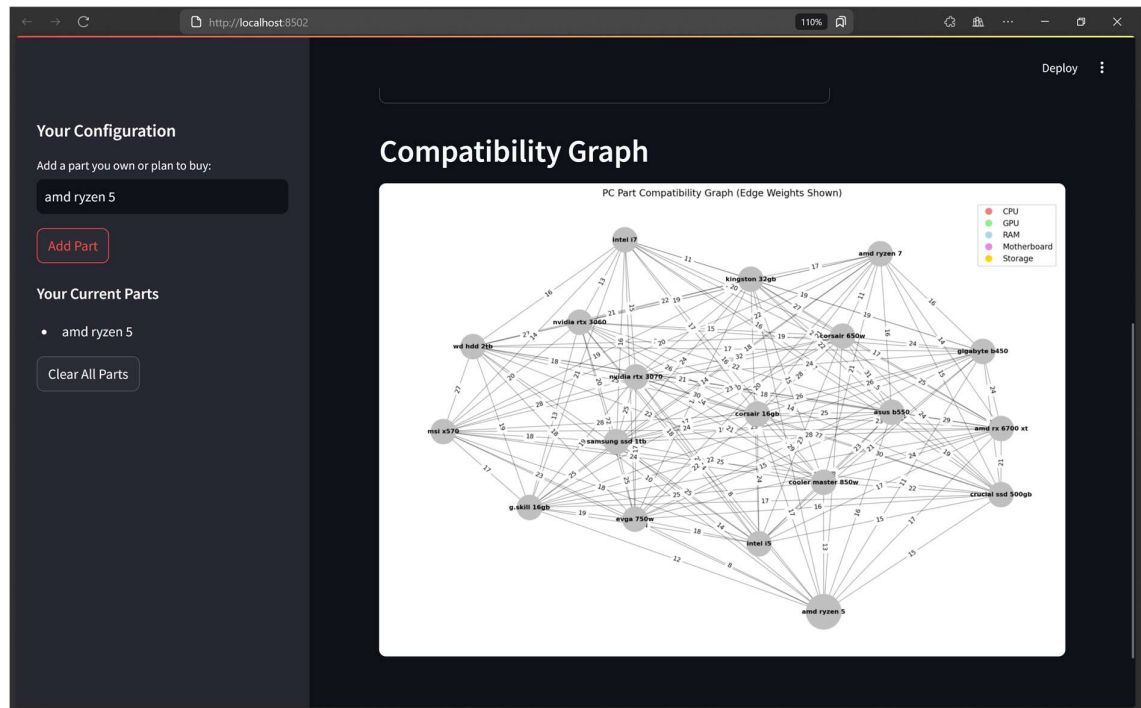


Figure 5.3: Network graph displaying frequently co-occurring e-waste components.

5.1.3 Economic Descriptive Modeling Module

Objective

The **Economic Descriptive Modeling Module** determines whether an identified component should be **resold or recycled** based on market trends, component conditions, and estimated profitability.

Implementation Details

1. Market Data Integration

- Real-time price data for identified components is **scraped from e-commerce platforms** such as **eBay and Amazon**.
- The system collects historical pricing trends for **resale value prediction**.

2. Resale Value Prediction

- A **Random Forest Regression model** is trained to predict resale value based on:
 - Component type and condition.
 - Brand reputation.
 - Demand and supply trends.

3. Recycling Cost Estimation

- Recycling costs are estimated using a cost model that includes:
 - **Processing costs** (e.g., dismantling and extraction).
 - **Labor costs**.
 - **Energy consumption** for recycling operations.

4. Comparative Analysis and Decision Support

- The system compares the **estimated resale value vs. recycling costs** and suggests the optimal action.
- A **decision table** (Figure 5.4) displays recommendations for each classified component.

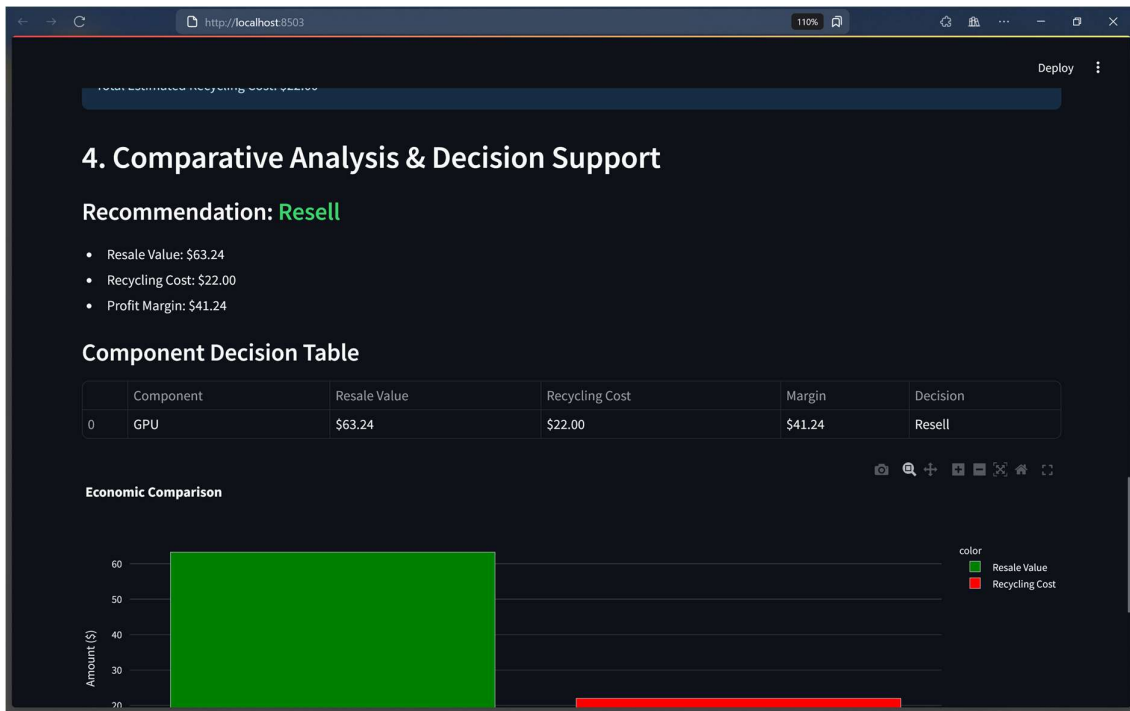


Figure 5.4: Decision table for resale vs. recycling cost analysis.

5.1.4 Front-End Module

Objective

The **Front-End Module** provides an **intuitive user interface** for interacting with the system, ensuring **seamless navigation and accessibility**.

Implementation Details

1. User Interface Design

- The UI is built using **Streamlit**, a lightweight framework ideal for rapid deployment of interactive applications.

2. Core Features

- The front-end includes the following sections:
 - Image Upload Interface** – Allows users to upload e-waste images (**Figure 5.1**).

2. **Component Classification Display** – Shows detected components and labels (**Figure 5.2**).
3. **Market Trends Dashboard** – Displays real-time price trends for electronic components (**Figure 5.5**).
4. **Final Decision Report** – Generates a downloadable report with component classifications, economic insights, and suggested actions (**Figure 5.6**).

Chapter 6

Conclusion & Future Enhancements

6.1 Conclusion

- **EcoSort Intelligence successfully automates e-waste classification and profit prediction using deep learning and market analysis.**
- **Key Achievements:**
 - High classification accuracy using ResNet and EfficientNet.
 - Efficient prediction of recycling vs. reuse profitability using Market Basket Analysis.
 - Reduced human errors and labor effort in sorting processes.
 - Enhanced decision-making for sustainable e-waste management.

6.2 Future Enhancements

- **Real-time Component Price Updates:**
 - Integrate API-based market tracking for real-time price evaluation.
 - Ensure up-to-date financial insights for recycling businesses.
- **Expanded Dataset for Training:**
 - Improve classification accuracy by incorporating more labeled e-waste images.
 - Utilize diverse datasets to enhance model robustness.
- **Cloud Deployment:**
 - Host the application on a scalable cloud platform for enhanced accessibility.
 - Enable real-time processing and classification for users globally.
- **Blockchain Integration:**
 - Ensure transparency and traceability in the e-waste supply chain.
 - Maintain a secure ledger for tracking component recycling history.
- **Automated Sorting Mechanism:**
 - Develop a robotic arm for physical e-waste segregation based on classification results.
 - Reduce manual intervention in sorting processes.

Reference

1. Abou Baker, N., Stehr, J., & Handmann, U. (2022). Transfer learning approach towards a smarter recycling. *Neural Computing and Applications*, 34(Issue), 12345–12360
2. Li, Haomiao, Zian Jin, and Sujatha Krishnamoorthy. "E-waste management using machine learning." Proceedings of the 6th International Conference on Big Data and Computing. 2021.
3. Abou Baker, Nermeen, Paul Szabo-Müller, and Uwe Handmann. "Transfer learning-based method for automated e-waste recycling in smart cities." *EAI endorsed transactions on smart cities* 5.16 (2021): e1-e1.
4. Sarkhoshkalat, Mohammad Mahdi, et al. "Circular Economy and the Recycling of E-Waste." *New Technologies for Energy Transition Based on Sustainable Development Goals: Factors Contributing to Global Warming*. Singapore: Springer Nature Singapore, 2024. 319-354.
5. Russo, Giorgio L. "Integrated Management of Electronic and Electric Waste (EEW) with the Application of Artificial Intelligence (AI): Future and Challenges." *Advanced Technologies for Solid, Liquid, and Gas Waste Treatment*. CRC Press, 2023. 23-40.
6. Selvakanmani, S., et al. "Optimizing E-waste management: Deep learning classifiers for effective planning." *Journal of Cleaner Production* 443 (2024): 141021.
7. Moradi, Rosita, et al. "Sustainable resilient E-waste management in London: A circular economy perspective." *Heliyon* 10.13 (2024).
8. Oise, Godfrey Perfectson, and Susan Konyeha. "Deep learning system for e-waste management." *Engineering Proceedings* 67.1 (2024): 66.
9. BAYDOĞAN, Merve PARLAK. "Deep Learning-Based Approaches for Electronic Waste Detection and Classification: A Step Toward Sustainable Recycling." *JOURNAL OF MATERIALS AND ELECTRONIC DEVICES* 5.1 (2024): 49-54.
10. Singh, Rahulkumar Sunil, et al. "Identification and Separation of E-Waste Components Using Modified Image Recognition Model Based on Advanced Deep Learning Tools." *Technology Innovation for the Circular Economy: Recycling, Remanufacturing, Design, Systems Analysis and Logistics* (2024): 115-127.

Appendix A1: Source Code

Graph:

```
import networkx as nx
import matplotlib.pyplot as plt
import pandas as pd
import pickle
from typing import List, Dict, Optional, Tuple
from collections import defaultdict

class EnhancedPCPartGraph:
    def __init__(self, data_file: str = "pc_market_data.csv"):
        self.graph = nx.Graph()
        self.data_file = data_file
        self.transactions = self.load_data()
        self.build_graph()
        self._initialize_category_weights()
        self._setup_compatibility_rules()

    def _initialize_category_weights(self):
        self.category_weights = {
            "CPU": 1.5, "GPU": 1.3, "RAM": 1.2,
            "Motherboard": 1.4, "Storage": 1.1,
            "PSU": 1.0, "Case": 1.0, "Cooler": 1.0, "Other": 1.0
        }

    def _setup_compatibility_rules(self):
        self.incompatible_pairs = [
            ("intel", "am[0-9] motherboard", "Intel CPUs require Intel-compatible motherboards"),
            ("amd", "lga[0-9]+", "AMD CPUs require AMD-compatible motherboards"),
            ("ddr4", "ddr[235] motherboard", "DDR4 RAM requires DDR4-compatible motherboard"),
            ("atx psu", "sfx case", "ATX PSUs don't fit in SFX cases"),
            ("itx motherboard", "atx case", "ITX motherboards may have mounting issues in ATX cases")
        ]
        self.special_compatible = [("ram", "ram"), ("ssd", "ssd"), ("hdd", "hdd")]
```

```

def load_data(self) -> List[List[str]]:
    try:
        df = pd.read_csv(self.data_file)
        df = df.dropna(subset=["PurchasedParts"])
        transactions = []
        for parts_list in df["PurchasedParts"]:
            try:
                parts = [
                    part.strip().strip("`\"'")
                    for part in parts_list.strip("[]").split(",")
                    if part.strip()
                ]
                if parts:
                    transactions.append(parts)
            except (AttributeError, TypeError):
                continue
        return transactions
    except Exception as e:
        print(f"Error loading data: {e}")
        return []

```

```

def build_graph(self):
    for transaction in self.transactions:
        cleaned_parts = [self._normalize_part_name(p) for p in transaction]
        for i in range(len(cleaned_parts)):
            for j in range(i+1, len(cleaned_parts)):
                part1, part2 = cleaned_parts[i], cleaned_parts[j]
                if not part1 or not part2 or part1 == part2:
                    continue
                if self.graph.has_edge(part1, part2):
                    self.graph[part1][part2]["weight"] += 1
                else:
                    self.graph.add_edge(part1, part2, weight=1)

def _normalize_part_name(self, part_name: str) -> str:
    if not isinstance(part_name, str):
        return ""
    return part_name.strip().lower()

```

```

def get_part_category(self, part_name: str) -> str:
    part_lower = self._normalize_part_name(part_name)
    category_patterns = {
        "CPU": ["ryzen", "core i[0-9]", "xeon", "cpu", "processor"],
        "GPU": ["rtx[ _]?[0-9]", "gtx[ _]?[0-9]", "radeon", "gpu", "graphics"],
        "RAM": ["ddr[0-9]", "ram", "memory"],
        "Motherboard": ["b[0-9]{3}", "x[0-9]{3}", "motherboard", "mainboard", "mobo"],
        "Storage": ["ssd", "hdd", "nvme", "m.2", "storage"],
        "PSU": ["power supply", "psu", "[0-9]+w"],
        "Case": ["case", "chassis", "tower", "atx", "itx", "matx"]
    }
    for category, patterns in category_patterns.items():
        if any(self._matches_pattern(part_lower, pattern) for pattern in patterns):
            return category
    return "Other"

def _matches_pattern(self, text: str, pattern: str) -> bool:
    try:
        import re
        return bool(re.search(pattern, text, re.IGNORECASE))
    except:
        return pattern.lower() in text.lower()

```

```

def get_recommendations(self, purchased_parts: List[str], top_n: int = 5, min_weight: int = 1) -> List[str]:
    """Returns only part names without scores"""
    scored_recs = self._get_scored_recommendations(purchased_parts, top_n, min_weight)
    return [part for part, _ in scored_recs]

def _get_scored_recommendations(self, purchased_parts: List[str], top_n: int, min_weight: int) -> List[Tuple[str, float]]:
    recommendations = defaultdict(float)
    purchased_normalized = [self._normalize_part_name(p) for p in purchased_parts]

    for part in purchased_parts:
        part_normalized = self._normalize_part_name(part)
        if part_normalized not in self.graph:
            continue
        for neighbor, data in self.graph[part_normalized].items():
            if (neighbor in purchased_normalized or
                data.get("weight", 0) < min_weight or
                not self.check_compatibility(part, neighbor)):
                continue
            category = self.get_part_category(neighbor)
            score = data["weight"] * self.category_weights.get(category, 1.0)
            recommendations[neighbor] += score

    return sorted(recommendations.items(), key=lambda x: x[1], reverse=True)[:top_n]

```

```

def check_compatibility(self, part1: str, part2: str) -> bool:
    part1_normalized = self._normalize_part_name(part1)
    part2_normalized = self._normalize_part_name(part2)
    for pattern1, pattern2 in self.special_compatible:
        if (self._matches_pattern(part1_normalized, pattern1) and
            self._matches_pattern(part2_normalized, pattern2)):
            return True
    for pattern1, pattern2, _ in self.incompatible_pairs:
        if (self._matches_pattern(part1_normalized, pattern1) and
            self._matches_pattern(part2_normalized, pattern2)) or \
            ((self._matches_pattern(part1_normalized, pattern2) and
              self._matches_pattern(part2_normalized, pattern1))):
            return False
    return True

def get_compatibility_reason(self, part1: str, part2: str) -> str:
    part1_normalized = self._normalize_part_name(part1)
    part2_normalized = self._normalize_part_name(part2)
    for pattern1, pattern2 in self.special_compatible:
        if (self._matches_pattern(part1_normalized, pattern1)) and \
            (self._matches_pattern(part2_normalized, pattern2)):
            return "Multiple instances of this component type are supported"
    for pattern1, pattern2, reason in self.incompatible_pairs:
        if ((self._matches_pattern(part1_normalized, pattern1)) and
            self._matches_pattern(part2_normalized, pattern2)) or \
            ((self._matches_pattern(part1_normalized, pattern2) and
              self._matches_pattern(part2_normalized, pattern1))):
            return reason
    return "Compatible based on available data"

```

```

def visualize_graph(self, highlight_parts: List[str] = None, figsize: Tuple[int, int] = (12, 8)):
    """Enhanced to show edge weights"""
    if not self.graph.nodes():
        print("Graph is empty - nothing to visualize")
        return

    plt.figure(figsize=figsize)
    pos = nx.spring_layout(self.graph, k=0.5, iterations=50)

    # Draw nodes and edges
    node_colors = [self._get_category_color(node) for node in self.graph.nodes()]
    node_sizes = [2000 if highlight_parts and node in highlight_parts else 1000 for node in self.graph.nodes()]
    edge_widths = 0.3

    nx.draw(self.graph, pos,
            with_labels=True,
            node_color=node_colors,
            node_size=node_sizes,
            edge_color="black",
            width=edge_widths,
            font_size=8,
            font_weight="bold")

    # Add edge weight labels
    edge_labels = {(u, v): f"{d['weight']}" for u, v, d in self.graph.edges(data=True)}
    nx.draw_networkx_edge_labels(self.graph, pos, edge_labels=edge_labels, font_size=8)

    self._add_category_legend()
    plt.title("PC Part Compatibility Graph (Edge Weights Shown)")
    plt.tight_layout()

```

```

    return plt.gcf()

def _get_category_color(self, category: str) -> str:
    color_map = {
        "CPU": "lightcoral", "GPU": "lightgreen", "RAM": "lightblue",
        "Motherboard": "violet", "Storage": "gold", "PSU": "orange",
        "Case": "lightgray", "Other": "silver"
    }
    return color_map.get(category, "silver")

def _add_category_legend(self):
    from matplotlib.lines import Line2D
    legend_elements = [
        Line2D([0], [0], marker='o', color='w', label='CPU', markerfacecolor='lightcoral', markersize=10),
        Line2D([0], [0], marker='o', color='w', label='GPU', markerfacecolor='lightgreen', markersize=10),
        Line2D([0], [0], marker='o', color='w', label='RAM', markerfacecolor='lightblue', markersize=10),
        Line2D([0], [0], marker='o', color='w', label='Motherboard', markerfacecolor='violet', markersize=10),
        Line2D([0], [0], marker='o', color='w', label='Storage', markerfacecolor='gold', markersize=10)
    ]
    plt.legend(handles=legend_elements, loc='upper right')

def save_state(self, filename: str = "pc_part_graph.pkl"):
    try:
        with open(filename, 'wb') as f:
            pickle.dump({'graph': self.graph, 'transactions': self.transactions}, f)
        print(f"Graph state saved to {filename}")
        return True
    except Exception as e:
        print(f"Error saving graph state: {e}")
        return False

```

```

def load_state(self, filename: str = "pc_part_graph.pkl"):
    try:
        with open(filename, 'rb') as f:
            data = pickle.load(f)
            self.graph = data['graph']
            self.transactions = data['transactions']
            print(f"Graph state loaded from {filename}")
            return True
    except Exception as e:
        print(f"Error loading graph state: {e}")
        self.graph = nx.Graph()
        self.transactions = []
        return False

def add_user_transaction(self, parts: List[str]):
    if not parts:
        return

    cleaned_parts = [self._normalize_part_name(p) for p in parts if p]
    self.transactions.append(cleaned_parts)

    for i in range(len(cleaned_parts)):
        for j in range(i+1, len(cleaned_parts)):
            part1, part2 = cleaned_parts[i], cleaned_parts[j]
            if part1 == part2:
                continue
            if self.graph.has_edge(part1, part2):
                self.graph[part1][part2]["weight"] += 1
            else:
                self.graph.add_edge(part1, part2, weight=1)

```

Main.py:

```
import torch
import torch.optim as optim
from torch.utils.data import DataLoader
from tqdm import tqdm
from preprocessor import CustomDataset, transform
from model import HybridModel

# Hyperparameters
BATCH_SIZE = 4
NUM_CLASSES = 8
EPOCHS = 1
LR = 1e-4
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"

# Load dataset
train_dataset = CustomDataset(image_dir="Dataset/valid", annotation_file="Dataset/valid/_annotations.coco.json", transform=transform)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, collate_fn=lambda x: tuple(zip(*x)))

# Load model
model = HybridModel(num_classes=NUM_CLASSES).to(DEVICE)
optimizer = optim.Adam(model.parameters(), lr=LR)
classification_loss_fn = torch.nn.CrossEntropyLoss()

# Training loop
for epoch in range(EPOCHS):
    model.train()
    epoch_loss = 0

    progress_bar = tqdm(train_loader, desc=f"Epoch [{epoch+1}/{EPOCHS}]", leave=True)

    for images, targets in progress_bar:
        images = [img.to(DEVICE) for img in images]
        targets = [{k: v.to(DEVICE) for k, v in t.items()} for t in targets]

        optimizer.zero_grad()

        class_output, detection_output = model(torch.stack(images), targets)

        # Compute losses
        loss_class = classification_loss_fn(
            class_output,
            torch.tensor([t["labels"][0] for t in targets], dtype=torch.long).to(DEVICE),
        )
        loss_detection = detection_output["loss_classifier"] + detection_output["loss_box_reg"] + \
            detection_output["loss_objectness"] + detection_output["loss_rpn_box_reg"]

        loss = loss_class + loss_detection
        loss.backward()
        optimizer.step()

        epoch_loss += loss.item()
        progress_bar.set_postfix(loss=f"{loss.item():.4f}")

    print(f"Epoch [{epoch+1}/{EPOCHS}] - Avg Loss: {epoch_loss / len(train_loader):.4f}")

print("Training Complete.")
```


Model.py:

```
import torch
import torch.nn as nn
import torchvision.models as models
from torchvision.models.detection import fasterrcnn_resnet50_fpn
from torchvision.models.detection.faster_rcnn import FastRCNNPredictor

class HybridModel(nn.Module):
    def __init__(self, num_classes):
        super(HybridModel, self).__init__()

        # EfficientNet as feature extractor
        self.efficient_net = models.efficientnet_b0(pretrained=True)
        self.efficient_net.classifier = nn.Identity()

        # ResNet for feature extraction
        self.resnet = models.resnet18(pretrained=True)
        self.resnet.fc = nn.Identity()

        # Faster R-CNN for object detection
        self.faster_rcnn = fasterrcnn_resnet50_fpn(pretrained=True)
        in_features = self.faster_rcnn.roi_heads.box_predictor.cls_score.in_features
        self.faster_rcnn.roi_heads.box_predictor = FastRCNNPredictor(in_features, num_classes)

        # Fully connected layer for classification (EfficientNet + ResNet)
        self.fc = nn.Linear(1280 + 512, num_classes) # EfficientNet_B0 (1280) + ResNet18 (512)

    def forward(self, x, targets=None):
        eff_features = self.efficient_net(x)
        res_features = self.resnet(x)
        combined_features = torch.cat((eff_features, res_features), dim=1)
```



```

        class_output = self.fc(combined_features)

        if self.training:
            assert targets is not None, "Targets should not be None in training mode"
            detection_output = self.faster_rcnn(x, targets)
        else:
            detection_output = self.faster_rcnn(x)

        return class_output, detection_output

# Example usage
if __name__ == "__main__":
    model = HybridModel(num_classes=10)
    sample_input = torch.randn(2, 3, 224, 224)
    sample_targets = [{"boxes": torch.tensor([[50, 50, 150, 150]]), "labels": torch.tensor([1])}] * 2

    class_out, detect_out = model(sample_input, sample_targets)
    print(class_out.shape, detect_out)

```

Preprocessor:

```

import torch
from torchvision import transforms
import torchvision.transforms.functional as F
import cv2
import os
import json
from torch.utils.data import Dataset

class CustomDataset(Dataset):
    def __init__(self, image_dir, annotation_file, transform=None):
        """
        Args:
            image_dir (str): Path to images folder.
            annotation_file (str): Path to COCO JSON annotations.
            transform (callable, optional): Image transformations.
        """
        self.image_dir = image_dir
        self.transform = transform

        with open(annotation_file, 'r') as f:
            self.annotations = json.load(f)

        self.image_ids = list(set(ann["image_id"] for ann in self.annotations["annotations"]))
        self.image_info = {img["id"]: img["file_name"] for img in self.annotations["images"]}

    def __len__(self):
        return len(self.image_ids)

    def __getitem__(self, idx):
        image_id = self.image_ids[idx]

```

```

        # Convert to tensors
        boxes = torch.as_tensor(boxes, dtype=torch.float32)
        labels = torch.as_tensor(labels, dtype=torch.int64)

        targets = {"boxes": boxes, "labels": labels}

        if self.transform:
            image = self.transform(image)

        return image, targets

# Define transforms
transform = transforms.Compose([
    transforms.ToPILImage(),
    transforms.Resize((320, 320)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

```

Train.py:

```

import torch
import torch.optim as optim
from torch.utils.data import DataLoader
from tqdm import tqdm
from preprocessor import CustomDataset, transform
from model import HybridModel

# Hyperparameters
BATCH_SIZE = 4
NUM_CLASSES = 8
EPOCHS = 1
LR = 1e-4
DEVICE = "cuda" if torch.cuda.is_available() else "cpu"
MODEL_SAVE_PATH = "model.pth"

# Load dataset
train_dataset = CustomDataset(image_dir="Dataset/valid", annotation_file="Dataset/valid/_annotations.coco.json", transform=transform)
train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True, collate_fn=lambda x: tuple(zip(*x)))

# Load model
model = HybridModel(num_classes=NUM_CLASSES).to(DEVICE)
optimizer = optim.Adam(model.parameters(), lr=LR)
classification_loss_fn = torch.nn.CrossEntropyLoss()

# Training loop
for epoch in range(EPOCHS):
    model.train()
    epoch_loss = 0
    correct = 0
    total = 0

    progress_bar = tqdm(train_loader, desc=f"Epoch [{epoch+1}/{EPOCHS}]", leave=True)

```

```

for images, targets in progress_bar:
    images = [torch.tensor(img, dtype=torch.float32).to(DEVICE) for img in images]
    targets = [[k: torch.tensor(v).to(DEVICE) if isinstance(v, torch.Tensor) else torch.tensor(v, dtype=torch.long).to(DEVICE) for k, v in t.items()] for t in targets]

    optimizer.zero_grad()

    class_output, detection_output = model(torch.stack(images), targets)

    # Compute losses
    loss_class = classification_loss_fn(
        class_output,
        torch.tensor([t["labels"][0] for t in targets], dtype=torch.long).to(DEVICE),
    )
    loss_detection = detection_output["loss_classifier"] + detection_output["loss_box_reg"] + \
        detection_output["loss_objectness"] + detection_output["loss_rpn_box_reg"]

    loss = loss_class + loss_detection
    loss.backward()
    optimizer.step()

    epoch_loss += loss.item()
    _, predicted = torch.max(class_output, 1)
    total += len(targets)
    correct += (predicted == torch.tensor([t["labels"][0] for t in targets], dtype=torch.long).to(DEVICE)).sum().item()

    progress_bar.set_postfix(loss=f"{loss.item():.4f}", accuracy=f"{100 * correct / total:.2f}%")

print(f"Epoch [{epoch+1}/{EPOCHS}] - Avg Loss: {epoch_loss / len(train_loader):.4f} - Accuracy: {100 * correct / total:.2f}%")

print("Training Complete.")
torch.save(model.state_dict(), MODEL_SAVE_PATH)

```

Test.py:

```

import streamlit as st
import torch
import torchvision.transforms as T
from PIL import Image, ImageDraw, ImageFont
import numpy as np
from model import HybridModel

# Load the custom model
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
MODEL_PATH = "model.pth" # Update this with your actual model path
NUM_CLASSES = 8 # Update according to your dataset

model = HybridModel(num_classes=NUM_CLASSES).to(device)
model.load_state_dict(torch.load(MODEL_PATH, map_location=device))
model.eval()

# Define the transformation
transform = T.Compose([
    T.Resize((320, 320)),
    T.ToTensor(),
    T.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
])

# Class label mapping (Modify based on your dataset)
CLASS_NAMES = {1: "Laptop", 2: "Motherboard", 3: "Battery", 4: "CPU", 5: "Other"}

def detect_objects(image):
    img_tensor = transform(image).unsqueeze(0).to(device) # Add batch dimension
    with torch.no_grad():
        _, predictions = model(img_tensor)

```

```

        predictions = {key: value.cpu().numpy() for key, value in predictions.items()}
        return predictions

    def draw_boxes(image, predictions, threshold=0.5):
        image = image.copy()
        draw = ImageDraw.Draw(image)
        try:
            font = ImageFont.truetype("arial.ttf", 20)
        except:
            font = ImageFont.load_default()

        for box, score, label in zip(predictions['boxes'], predictions['scores'], predictions['labels']):
            if score > threshold:
                box = [int(x) for x in box]
                class_name = CLASS_NAMES.get(int(label), f"Class {int(label)}")
                draw.rectangle(box, outline="red", width=3)
                draw.text((box[0], box[1] - 10), f"{class_name}: {score:.2f}", fill="red", font=font)

        return image

# Streamlit UI
st.title("Object Detection using HybridModel (EfficientNet + ResNet + Faster R-CNN)")
uploaded_file = st.file_uploader("Upload an image", type=["jpg", "png", "jpeg"])

if uploaded_file is not None:
    image = Image.open(uploaded_file).convert("RGB")
    st.image(image, caption="Uploaded Image", use_column_width=True)

    st.write("Detecting objects...")
    predictions = detect_objects(image)

    if len(predictions.get('boxes', [])) == 0:
        st.write("No objects detected.")
    else:
        output_image = draw_boxes(image, predictions)
        st.image(output_image, caption="Detected Objects", use_column_width=True)

        st.write("Detected Classes and Scores:")
        for label, score in zip(predictions['labels'], predictions['scores']):
            if score > 0.5:
                class_name = CLASS_NAMES.get(int(label), f"Class {int(label)}")
                st.write(f"{class_name}: {score:.2f}")

```

Streamlit:

```

import streamlit as st
from graph_rec import EnhancedPCPartGraph
from Scraper import PCMarketScraper
import pandas as pd
import time
import matplotlib.pyplot as plt

def main():
    st.set_page_config(page_title="PC Part Advisor", layout="wide")

    # Initialize session state
    if 'purchased_parts' not in st.session_state:
        st.session_state.purchased_parts = []
    if 'recommendations' not in st.session_state:
        st.session_state.recommendations = []

    # Initialize components
    pc_graph = EnhancedPCPartGraph()
    scraper = PCMarketScraper(headless=True)

    st.title("🖥️ PC Part Compatibility Advisor")
    st.markdown("""
    Get intelligent recommendations for PC parts based on compatibility and market trends.
    """)

    # Sidebar for user input
    with st.sidebar:
        st.header("Your Configuration")

        # Part input
        new_part = st.text_input("Add a part you own or plan to buy:")
        if st.button("Add Part") and new_part:
            if new_part not in st.session_state.purchased_parts:

```

```

        st.session_state.purchased_parts.append(new_part)
        st.success(f"Added {new_part} to your parts list!")
        time.sleep(1)
        st.rerun()

# Display current parts
if st.session_state.purchased_parts:
    st.subheader("Your Current Parts")
    for part in st.session_state.purchased_parts:
        st.write(f"- {part}")

    if st.button("Clear All Parts"):
        st.session_state.purchased_parts = []
        st.rerun()

# Main content area
col1, col2 = st.columns([2, 1])

with col1:
    st.header("Recommendations")

    if st.session_state.purchased_parts:
        # Get recommendations
        st.session_state.recommendations = pc_graph.get_recommendations(
            st.session_state.purchased_parts,
            top_n=5
        )

    if st.session_state.recommendations:
        st.success("Here are our top recommendations for your build:")
        for i, rec in enumerate(st.session_state.recommendations, 1):

```

```

            with st.expander(f"#{i}: {rec}"):
                # Get compatibility status
                compatible = all(pc_graph.check_compatibility(rec, p)
                                for p in st.session_state.purchased_parts)

                st.markdown(f"""
                - **Category**: {pc_graph.get_part_category(rec)}
                - **Compatibility**: {"✅ Compatible" if compatible else "❌ Incompatible"}
                - **Reason**: {pc_graph.get_compatibility_reason(rec, st.session_state.purchased_parts)}
                """)

                if compatible and st.button(f"Add {rec} to your build", key=f"add_{rec}"):
                    st.session_state.purchased_parts.append(rec)
                    pc_graph.add_user_transaction(st.session_state.purchased_parts)
                    st.success(f"Added {rec} to your build!")
                    time.sleep(1)
                    st.rerun()
                else:
                    st.warning("No recommendations found. Try adding more parts.")

with col2:
    st.header("Market Trends")

    if st.button("Refresh Market Data"):
        with st.spinner("Fetching latest market trends..."):
            if scraper.update_market_data():
                st.success("Market data updated!")
            else:
                st.error("Failed to update market data")

```

```

    try:
        market_data = pd.read_csv("pc_market_data.csv")
        st.dataframe(market_data.head(10),
                     height=300,
                     use_container_width=True)
    except FileNotFoundError:
        st.warning("No market data available. Click 'Refresh' to fetch.")

# Graph visualization section
st.header("Compatibility Graph")
if st.session_state.purchased_parts:
    pc_graph.visualize_graph(st.session_state.purchased_parts)
    st.pyplot(plt.gcf())
else:
    st.info("Add parts to visualize compatibility relationships")

if __name__ == "__main__":
    main()

```

Scraper.py:

```

from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.chrome.service import Service
from selenium.webdriver.chrome.options import Options
from webdriver_manager.chrome import ChromeDriverManager
import pandas as pd
import time
from typing import List

class PCMarketScraper:
    def __init__(self, headless: bool = True):
        self.options = Options()
        if headless:
            self.options.add_argument("--headless")
            self.options.add_argument("--disable-gpu")
            self.options.add_argument("--no-sandbox")
            self.options.add_argument("--disable-dev-shm-usage")

    def scrape_newegg(self, url: str = "https://www.newegg.com/todays-best-deals", max_items: int = 15) -> List[str]:
        """Scrape Newegg for popular PC parts"""
        driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()),
                                options=self.options)

        parts = []

        try:
            driver.get(url)
            time.sleep(3)

            elements = driver.find_elements(By.CSS_SELECTOR, ".item-title")
            parts = [elem.text.strip() for elem in elements[:max_items]] if elements else []

        except Exception as e:
            print(f"Error scraping Newegg: {e}")

```



```

        finally:
            driver.quit()

        return parts

def scrape_ebay(self, url: str = "https://www.ebay.com/deals/computers-tablets", max_items: int = 15) -> List[str]:
    """Scrape eBay for popular PC parts"""
    driver = webdriver.Chrome(service=Service(ChromeDriverManager().install()),
                             options=self.options)

    parts = []

    try:
        driver.get(url)
        time.sleep(3)

        elements = driver.find_elements(By.CSS_SELECTOR, ".ebayui-ellipsis-2")
        parts = [elem.text.strip() for elem in elements[:max_items]] if elements else []

    except Exception as e:
        print(f"Error scraping eBay: {e}")
    finally:
        driver.quit()

    return parts

def scrape_market_data(self) -> pd.DataFrame:
    """Scrape both Newegg and eBay and return combined DataFrame"""
    newegg_parts = self.scrape_newegg()
    ebay_parts = self.scrape_ebay()

```

```

    data = []
    if newegg_parts:
        data.extend([("Newegg", part) for part in newegg_parts])
    if ebay_parts:
        data.extend([("eBay", part) for part in ebay_parts])

    return pd.DataFrame(data, columns=["Source", "PC Part"])

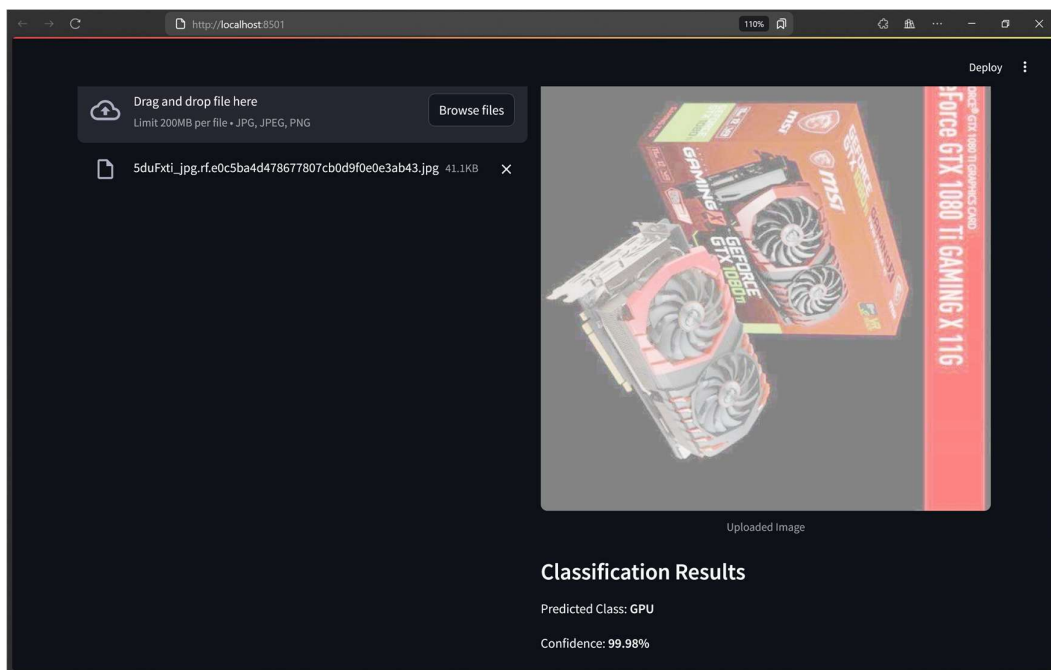
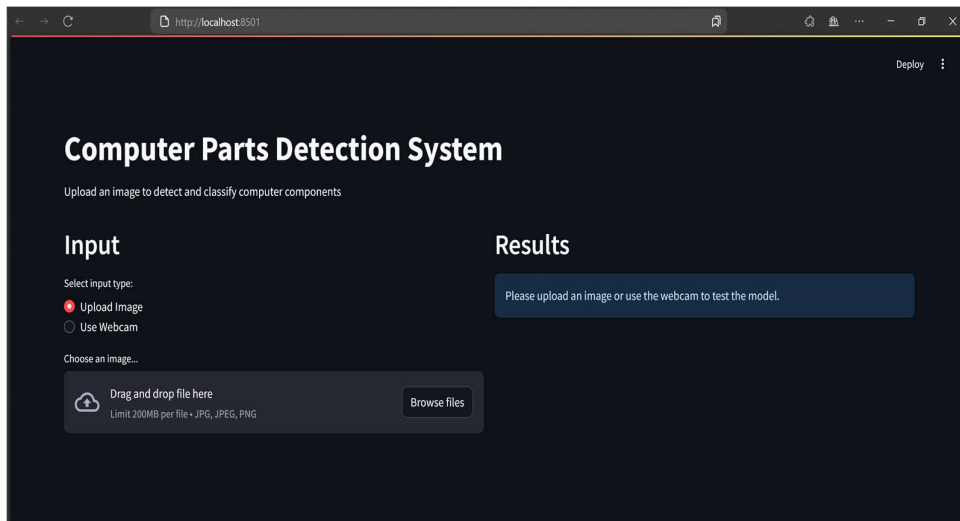
def update_market_data(self, filename: str = "pc_market_data.csv"):
    """Update the market data CSV file with new scraped data"""
    try:
        new_data = self.scrape_market_data()
        if not new_data.empty:
            # Read existing data and append new data
            try:
                existing_data = pd.read_csv(filename)
                combined_data = pd.concat([existing_data, new_data]).drop_duplicates()
            except FileNotFoundError:
                combined_data = new_data

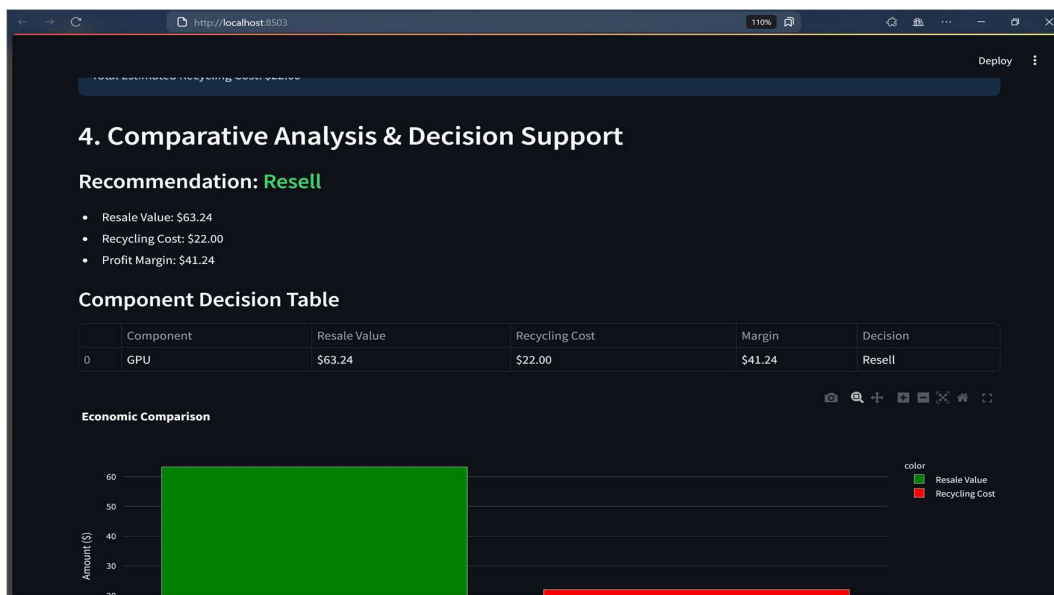
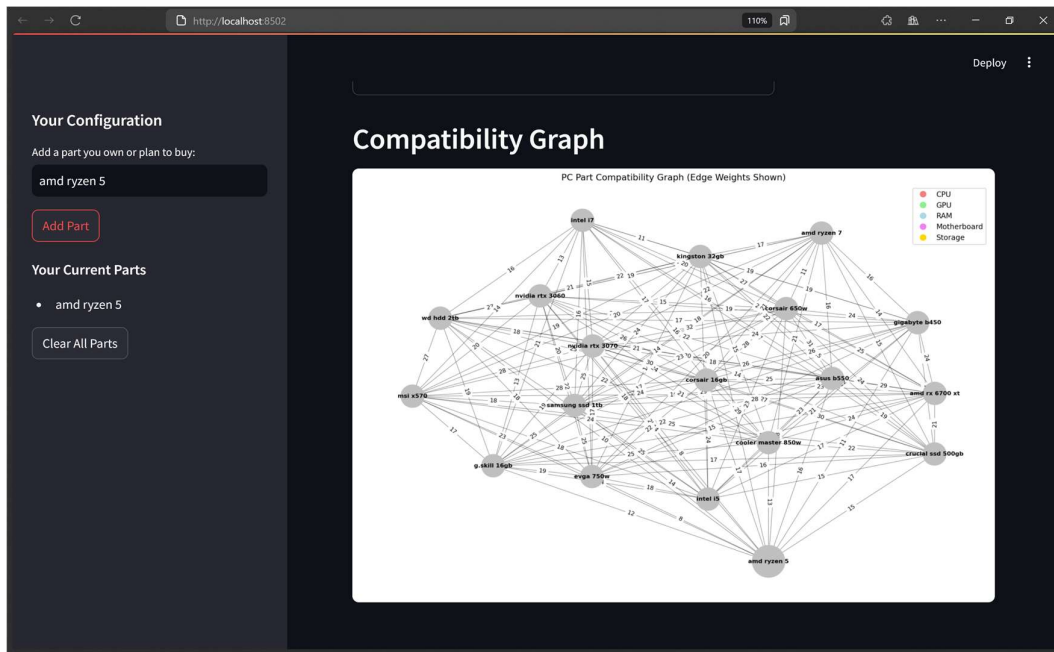
            combined_data.to_csv(filename, index=False)
            print(f"Market data updated in {filename}")
            return True
    except Exception as e:
        print(f"Error updating market data: {e}")

    return False

```


Appendix A2:





Technical Biography



Akshay Prashad MK (210171601007), was born in the year 2003 in Chennai, Tamil Nadu, India. He completed his higher secondary education in 2021. He is currently pursuing a B. Tech Artificial Intelligence and Data Science at the school of Computer information and mathematical sciences at B.S. Abdur Rahman Crescent Institute of Science and Technology which is located at Vandalur, Chennai, India. His interests lie in the area of Data Science, Machine Learning and such. The email for communication is Akshayprasad.Psm@gmail.com and his contact number is 7904508275



Kamal.V (210171601021), was born in the year 2004 in Chennai, Tamil Nadu, India. He completed his higher secondary education in 2021. He is currently pursuing a B. Tech Artificial Intelligence and Data Science at the school of Computer information and mathematical sciences at B.S. Abdur Rahman Crescent Institute of Science and Technology which is located at Vandalur, Chennai, India. His interests lie in the area of Data Science, Machine Learning and such. The email for communication is kamalvenkateswaran17@gmail.com and his contact number is 6369834039.

